

Segmentação por Textura (TA01)

Millena Suiani Costa
Departamento de Informática
Universidade Federal do Paraná – UFPR
Curitiba, Brasil

<https://github.com/millenaSui/VisaoComputacional-T1>

I. INTRODUÇÃO

O presente trabalho consiste na implementação de um *script* de segmentação por filtros de textura aplicados em uma coleção de imagens, visando distinguir suas informações visuais de forma puramente matemática. Utilizou-se a linguagem *Python* juntamente com as bibliotecas *NumPy* e *OpenCV*, processando um conjunto de imagens fotográficas de autoria própria com dimensões variadas, devidamente padronizadas para escala de cinza e resolução de 512×512 *pixels*.

II. DATASET E PRÉ-PROCESSAMENTO

O *dataset* foi composto por ao menos 32 imagens fotográficas de autoria própria (armazenadas no diretório `./images/raw`). Para atender à padronização estipulada no enunciado e garantir um comportamento determinístico dos filtros, implementou-se um *pipeline* de pré-processamento automatizado.

Decisão de Projeto (Recorte e Redimensionamento): Como as imagens originais apresentavam proporções e resoluções distintas, a estratégia adotada para evitar distorções geométricas foi a extração do maior quadrado central possível (determinado pela menor aresta da imagem original). Em seguida, o recorte foi redimensionado para a resolução exata de 512×512 *pixels* utilizando a interpolação de área (`cv2.INTER_AREA`), recomendada para reduções de escala por preservar propriedades estatísticas locais, culminando na conversão para escala de cinza.

Adicionalmente, visando atender à exigência de processar um conjunto robusto de imagens de forma eficiente, o fluxo incorporou paralelismo de processos via módulo `concurrent.futures` (*ProcessPoolExecutor*), automatizando a conversão e o armazenamento no diretório de destino (`./images/processed`).



Figura 1. Imagem Original



Figura 2. Imagem Processada

III. BANCO DE FILTROS E ESTRATÉGIA DE ESCALAS

O enunciado estipulava a construção de um conjunto de filtros de textura cobrindo as orientações horizontal, vertical, 45° , 135° e circular, processados em 3 escalas, resultando em descritores de 24 dimensões por região.

Decisão de Projeto (Abordagem Multi-escala): Optou-se pela construção dos filtros em 3 escalas distintas ($\sigma \in \{0.5, 1.0, 2.0\}$), evitando a perda irreversível de resolução espacial inerente às reduções sucessivas de tamanho da imagem, garantindo que a extração de características ocorra em todo o grid original de 512×512 *pixels*.

O banco totalizou exatamente 24 filtros (8 tipos $\times$ 3 escalas), estruturados da seguinte forma em cada escala:

- **4 Filtros de Bordas (Direcionais):** Utilizou-se o filtro de Gabor com componente antissimétrica para os ângulos de 0° , 45° , 90° e 135° .
- **3 Filtros de Barras (Direcionais):** Utilizou-se o filtro de Gabor com componente simétrica para os ângulos de 0° , 45° e 90° . A omissão deliberada do ângulo de 135° nesta categoria ocorreu devido a necessidade de manter uma proporção de 8 filtros por escala, satisfazendo a restrição matemática de 24 dimensões no vetor final ($8 \times 3 = 24$).
- **1 Filtro Circular:** Implementou-se manualmente uma máscara em formato de disco baseada na distância radial ($r^2 \leq \text{raio}^2$), com raio parametrizado pela escala (σ), normalizada por sua soma unitária, atuando como um filtro de suavização circular.

O tamanho dos *kernels* em cada escala foi determinado dinamicamente pela regra `int(6 * sigma) | 1`, assegurando dimensões ímpares para a simetria de centralização nas operações de convolução (`cv2.filter2D`).

IV. CARACTERÍSTICAS POR JANELA E AGRUPAMENTO N-DIMENSIONAL

Com o banco de 24 filtros aplicado por meio de convoluções em ponto flutuante de 32 *bits* (evitando estouros numéricos), o mapeamento espacial e a vetorização foram executados dividindo-se a imagem em blocos não sobrepostos de 16×16 *pixels*. Para cada bloco, extraiu-se a média do valor absoluto das respostas de filtragem, gerando o vetor descritivo $v \in \mathbb{R}^{24}$.

Decisão de Projeto (Normalização e Agrupamento por Distância Euclidiana): Para agrupar as regiões semelhantes

conforme solicitado, implementou-se um algoritmo de agrupamento sequencial (*leader-follower*) baseado na distância euclidiana N-dimensional.

- 1) **Normalização Estatística:** Antes do cálculo das distâncias, os vetores de características passaram por uma normalização de média zero e desvio padrão unitário, sendo crítica por impedir que filtros com respostas de maior magnitude numérica dominem matematicamente o cálculo da distância em detrimento de texturas mais sutis.
- 2) **Dinâmica de Agrupamento:** Os blocos foram iterativamente comparados aos protótipos de grupos existentes via norma euclidiana (`np.linalg.norm`). Se a distância estivesse abaixo do limiar estipulado (4.0), o bloco integrava o grupo com atualização dinâmica de seu centroide. Caso excedesse o limite e o teto de grupos (`MAX_GRUPOS = 4`) não tivesse sido atingido, um novo grupo era instanciado; caso contrário, o bloco era alocado ao grupo representativo mais próximo.

Por fim, a tradução dos resultados numéricos para o espaço visual consistiu na coloração dos blocos 16×16 correspondentes a cada categoria utilizando quatro cores distintas (Azul, Amarelo, Verde e Vermelho). Uma operação de sobreposição ponderada (`*overlay*` com peso de 0.6 para a imagem original em cinza e 0.4 para o mapa de cores segmentado) permitiu a inspeção visual clara e interpretável das regiões texturizadas segmentadas.

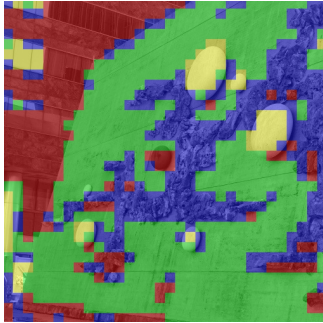


Figura 3. Exemplo 1 (seg_03.JPG)

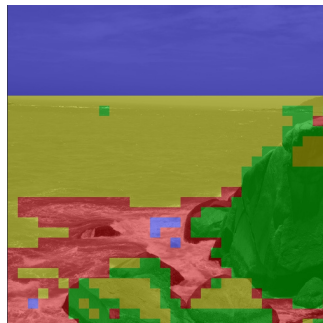


Figura 4. Exemplo 2 (seg_21.JPG)

V. ANÁLISE DE HIPERPARÂMETROS E IMPACTO DE MUDANÇAS ESTRUTURAIS

O desempenho do sistema de segmentação por textura e a qualidade das regiões delimitadas são diretamente influenciados por escolhas de hiperparâmetros configurados no código, com destaque para o tamanho da janela espacial (`WINDOW_SIZE`). Modificações nesses parâmetros alteram substancialmente o comportamento do algoritmo e afetam o compromisso entre resolução espacial e robustez estatística.

No código implementado, o parâmetro está fixado em blocos não sobrepostos de 16×16 pixels. O ajuste deste valor acarretou trade-offs importantes na representação visual dos testes:

- **Redução de Janela (Ex: 2×2):** Oferece maior resolução espacial, permitindo contornos e bordas mais precisos. No entanto, por analisar poucos pixels por vez, a média estatística fica muito sensível a ruídos locais. Isso gera instabilidade no vetor de características e prejudica o agrupamento, fazendo com que regiões de texturas diferentes se misturem incorretamente. As figuras abaixo ilustram a perda de contexto global ao comparar o tamanho de janela padrão com um tamanho de janela menor que o padrão:

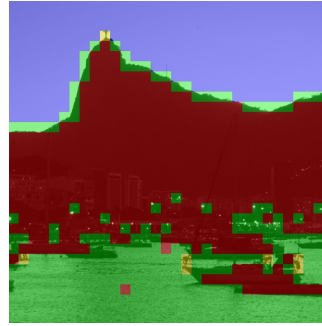


Figura 5. Paisagem com Janela de Tamanho 16x16

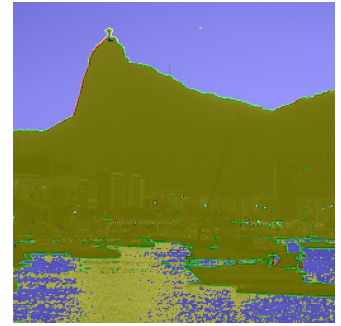


Figura 6. Paisagem com Janela de Tamanho 2x2

- **Aumento de Janela (Ex: 64×64):** Melhora a estabilidade estatística e suaviza o agrupamento, pois a média dos filtros é calculada sobre uma área ampla, eliminando ruídos pontuais. Em contrapartida, reduz severamente a resolução espacial, mascarando detalhes finos e borrando as transições entre texturas distintas. As figuras abaixo demonstram esse efeito ao proporcionar uma comparação entre a janela com o tamanho utilizado por padrão com uma janela de tamanho maior que o padrão:

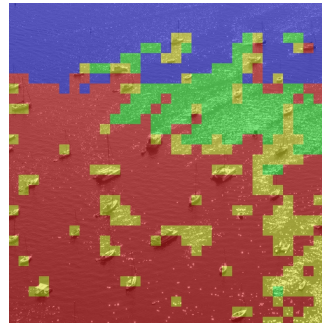


Figura 7. Paisagem com Janela de Tamanho 16x16

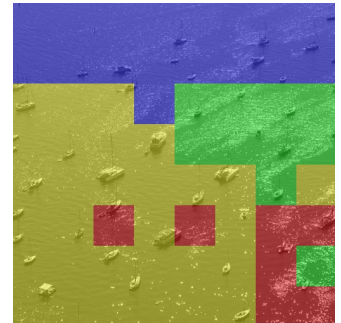


Figura 8. Paisagem com Janela de Tamanho 64x64